

Anti-patrones XP

Contexto

Los anti-patrones XP son formas incorrectas de adoptar las prácticas que producen los problemas opuestos a los que XP intenta resolver. Cada anti-patrón tiene una forma de manifestarse que puede parecer razonable bajo presión de tiempo o de velocidad, lo que los hace especialmente peligrosos.

El docente identifica cuatro anti-patrones principales. Este documento describe cada uno, explica cómo se manifiesta en un proyecto real y documenta cómo el proyecto los evita explícitamente.

Los cuatro anti-patrones

Anti-patrón 1: TDD sin cobertura significativa

Descripción. Adoptar el ciclo Red-Green-Refactor como formalidad: se escribe una prueba antes del código (cumpliendo el procedimiento visible), pero la prueba solo cubre el caso feliz más simple o un caso trivial que no representa la lógica de negocio real. El equipo puede reportar “usamos TDD” mientras la lógica crítica del sistema no tiene pruebas que la protejan.

Cómo se manifiesta. El developer escribe una prueba que verifica que `sumar(2, 2)` devuelve 4 antes de escribir la función `sumar`, pero no escribe pruebas que cubran los casos de borde del algoritmo de scoring de EP04: perfiles con afinidades empatadas entre múltiples carreras, perfiles con respuestas incompletas, o resultados cuando solo hay una carrera disponible.

El problema real. El motor de recomendación de este proyecto (EP04-US01, 8 puntos de historia) es el componente de mayor criticidad. Si TDD se aplica mecánicamente sin cubrir los casos que representan la lógica de negocio, el algoritmo puede producir resultados incorrectos con perfiles reales que no se probaron durante el desarrollo. El estudiante recibe una recomendación equivocada, que es el fallo funcional más grave del sistema.

Cómo lo evita el proyecto. El criterio DoD XP-1 no dice “tiene pruebas unitarias”: dice “tiene pruebas unitarias que validan los casos principales”. La distinción es operacional. HT-05 (suite de pruebas EP04, sprint 5) tiene como criterio de aceptación explícito que los casos límite del modelo estén cubiertos: perfiles con afinidades iguales, perfiles con respuestas incompletas, carreras sin correspondencia. El SM (Coach) verifica que las pruebas cubren casos reales, no solo el camino feliz.

Anti-patrón 2: Pair Programming como supervisión

Descripción. Usar Pair Programming como mecanismo de supervisión de un desarrollador por otro, en lugar de como colaboración horizontal entre pares. La forma más común es asignar a un senior como “navigator permanente” sobre

un junior, convirtiendo la sesión en una revisión de código en tiempo real bajo presión.

Cómo se manifiesta. Un developer con más experiencia en un módulo específico actúa como navigator en todas las sesiones de Pair Programming de ese módulo, porque “conoce mejor el código”. El developer menos familiarizado siempre es el driver. Los roles no rotan. La dinámica es de evaluación, no de colaboración.

El problema real. Pair Programming como supervisión destruye el valor de la práctica. El conocimiento técnico sigue concentrado en el mismo developer (el navigator permanente), que ahora además tiene el overhead de supervisar en lugar de producir. La comunicación —el valor XP que Pair Programming debe materializar— no se produce: el driver ejecuta instrucciones del navigator en lugar de pensar junto a él.

En un equipo de tres developers senior de nivel equivalente (Cruz, Chen, Guzmán), este anti-patrón es especialmente fácil de evitar: no hay jerarquía técnica que justifique la asignación permanente de roles. Pero puede aparecer cuando un developer tiene más contexto sobre un módulo específico (por ejemplo, quien implementó EP04 originalmente).

Cómo lo evita el proyecto. Los valores XP de Comunicación y Respeto definen Pair Programming como colaboración horizontal, no supervisión. Los pares se rotan deliberadamente: la rotación es un mecanismo explícito de distribución de conocimiento, no una consideración secundaria. Si un developer tiene más contexto sobre un módulo, el objetivo del par es precisamente transferir ese contexto, lo que requiere que el driver menos familiarizado piense activamente, no que ejecute instrucciones.

Anti-patrón 3: Refactor como tarea aparte

Descripción. Acumular deuda técnica durante los sprints con la intención de limpiarla en un sprint de refactoring al final del proyecto. Se trata el refactoring como una fase separada, no como una actividad continua integrada en cada historia.

Cómo se manifiesta. El equipo entrega historias funcionales con código que saben que está duplicado, mal estructurado o que viola los estándares de HT-02. La justificación es “lo refactorizamos en el sprint final” o “cuando tengamos más tiempo”. Al llegar al sprint 8, no hay capacidad para el refactoring porque el sprint final está cargado con EP06 (panel de resultados, consolidación) y con la estabilización para la entrega del 22 de abril 2027.

El problema real. La deuda técnica acumulada a lo largo de 7 sprints no se puede resolver en 1 o 2 sprints sin riesgo de introducir regresiones. Además, el código con deuda técnica hace que cada historia nueva tome más tiempo: el developer navega alrededor de la deuda para no romper nada, acumulando

más deuda en el proceso. La frase del docente aplica directamente: “La deuda técnica no se ve hasta que paraliza el proyecto.”

Cómo lo evita el proyecto. El refactoring está integrado en el DoD (criterio XP-3: sin deuda técnica deliberada). No es una tarea al final del proyecto: es la condición de cierre de cada historia. HT-04 (refactorización EP01, sprint 3) y HT-06 (refactorización EP04, sprint 6) son historias planificadas explícitamente para momentos óptimos del proyecto, no un sprint de limpieza al final. La regla del 20 % de capacidad para historias técnicas garantiza que el refactoring continuo tiene espacio en la planificación de cada sprint.

Anti-patrón 4: Adoptar Scrum sin prácticas XP

Descripción. Gestionar el proceso con Scrum (sprints, ceremonias, roles, artefactos) pero construir el código sin disciplina técnica. El equipo tiene Sprint Planning, Daily Scrum, Sprint Review y Retrospectiva, pero no tiene TDD, no tiene CI, no hace Pair Programming y no refactoriza. Scrum gestiona bien el proceso; el código se deteriora sprint tras sprint sin que nadie lo vea hasta que es demasiado tarde.

Cómo se manifiesta. El tablero del proyecto muestra historias que avanzan de columna en columna. La velocidad de los primeros sprints parece buena. Pero en el sprint 5 o 6, cuando EP04 y EP05 requieren construir sobre el código de EP01-EP03, los developers descubren que las integraciones no funcionan como esperaban, que hay duplicación que hace difícil razonar sobre el comportamiento del sistema y que no hay pruebas que les digan qué cambió cuando algo deja de funcionar.

Este es el anti-patrón que el docente identifica como el más común según el material del curso. La versión extrema es la que documenta la frase “La calidad no es solo visual”: un equipo que aprueba módulos visualmente en el Sprint Review sin verificar la funcionalidad real, porque las pruebas no existen y el CI no está configurado.

El problema real. Scrum hace el proceso visible pero no hace visible la calidad técnica del código. Sin TDD, sin CI y sin Refactoring, el proceso puede verse ordenado mientras el producto se deteriora. La deuda técnica acumulada en los primeros sprints hace que los sprints posteriores sean más lentos, no más rápidos, porque cada nueva historia tiene que navegar alrededor de código frágil y no probado.

Cómo lo evita el proyecto. XP es explícitamente la capa técnica del proyecto. No es un componente opcional: está integrado en el DoD (criterios XP-1, XP-2, XP-3), en el backlog (HT-01 a HT-07), en el flujo del tablero (columna Code Review) y en las métricas de calidad. El SM (Mathew Cordero) asume el rol de Coach XP de forma complementaria al de SM: es responsable de que las prácticas XP se apliquen con rigor, no solo que el proceso Scrum funcione correctamente.

El anti-patrón más común según el docente

El anti-patrón que el docente identifica como el más común es el número 4: adoptar Scrum sin prácticas XP. La razón es que Scrum es más visible que XP. Las ceremonias de Scrum son eventos con fecha, horario y participantes definidos: son difíciles de ignorar. Las prácticas de XP —TDD, CI, Refactoring— son hábitos de trabajo individuales y del equipo que no tienen un evento formal que las active.

Bajo presión de velocidad, es fácil que un equipo justifique omitir TDD en una historia porque “la lógica es sencilla” o posponer la configuración de CI porque “hay historias funcionales más urgentes”. Cada omisión individual parece razonable; el patrón acumulado produce código sin red de seguridad que paraliza el proyecto en el momento menos conveniente.

En este proyecto, los mecanismos que previenen este anti-patrón son operacionales, no declarativos: el DoD no permite cerrar historias sin CI verde, el backlog tiene historias técnicas con estimación y prioridad explícita, y el SM tiene el rol de Coach XP que hace su responsabilidad verificar que las prácticas se aplican.